

Lab 9

[valid 2024-2025]

Persistence

Continue the application from [lab 8](#), creating an object-oriented model and using JPA (Java Persistence API) in order to communicate with the relational database.

The main specifications of the application are:

Compulsory (1p)

- Create a *persistence unit* (use EclipseLink or Hibernate or other JPA implementation). Verify the presence of the *persistence.xml* file in your project. Make sure that the driver for EclipseLink or Hibernate was added to to your project classpath (or add it yourself).
 - Define the entity classes for your model (at least one) and put them in a dedicated package. You may use the IDE support in order to generate entity classes from database tables.
 - Create a *singleton* responsible with the management of an *EntityManagerFactory* object.
 - Define *repository* classes for your entities (at least one). They must contain the following methods:
 - *create* - receives an entity and saves it into the database;
 - *findById* - returns an entity based on its primary key;
 - *findByName* - returns a list of entities that match a given name pattern. Use a *named query* in order to implement this method.
 - Test your application.
-

Homework (2p)

- Create all entity classes and repositories. Implement properly the *one-to-many* and *many-to-many* relationships.
 - Create a generic *AbstractRepository* using *generics* in order to simplify the creation of the *repository* classes. You may take a look at the [CrudRepository](#) interface from Spring Framework.
 - Use the Java [logger](#) in order to log the exceptions and the execution time of your JPQL statements (you should write the log messages both on screen and in a file).
-

Bonus (2p)

- Implement both the JDBC and JPA data access classes and use an *AbstractFactory* in order to create the DAO objects (the repositories).
The application will use JDBC or JPA depending on a specified parameter stored in a *.properties* file.
- Assume each city has a new property, its *population*.
Use a constraint solver, such as [Choco solver](#), [OptaPlanner](#) or [ORTools](#), in order to find a set of cities having names that start with the same letter, the total sum of their population is between two given bounds and they are from different countries.

Resources

- [Slides](#)
- [JPA Tutorial](#)
- [Java EE Tutorial: Persistence](#)
- [Java Persistence Performance](#)